Technical Foray + Database Design

WhatToDo NYC

Cecilia Anna Anany
CS Capstone

---

1 Purpose

WhatToDo NYC is a context-aware discovery and recommendation application for New York City. It helps users decide what to do by combining:

- external place metadata (Google Places / maps listings)
- user-generated signals (reviews, ratings, authenticity votes, tags)
- real-time context (hours open/closed, weather, budget, group size)

The system returns a short ranked list with brief explanations ("why this").

2 Tech Stack

- **Frontend**: Next.js + Tailwind CSS, Google Maps JS SDK for visualization
- **Backend**: FastAPI (Python), SQLAlchemy, Pydantic, JWT auth, role-based access
- **Database**: PostgreSQL (Supabase)
- **External Services**: Google Places API, AccuWeather API
- **Optional**: LLM service for intent extraction + explanation phrasing (non-critical; deterministic fallback)

3 System Architecture

4 Primary User Flows

Explorer (Customer)

User enters mood/keywords, budget, group size, and indoor/outdoor preference. The app returns top recommendations with brief rationales.

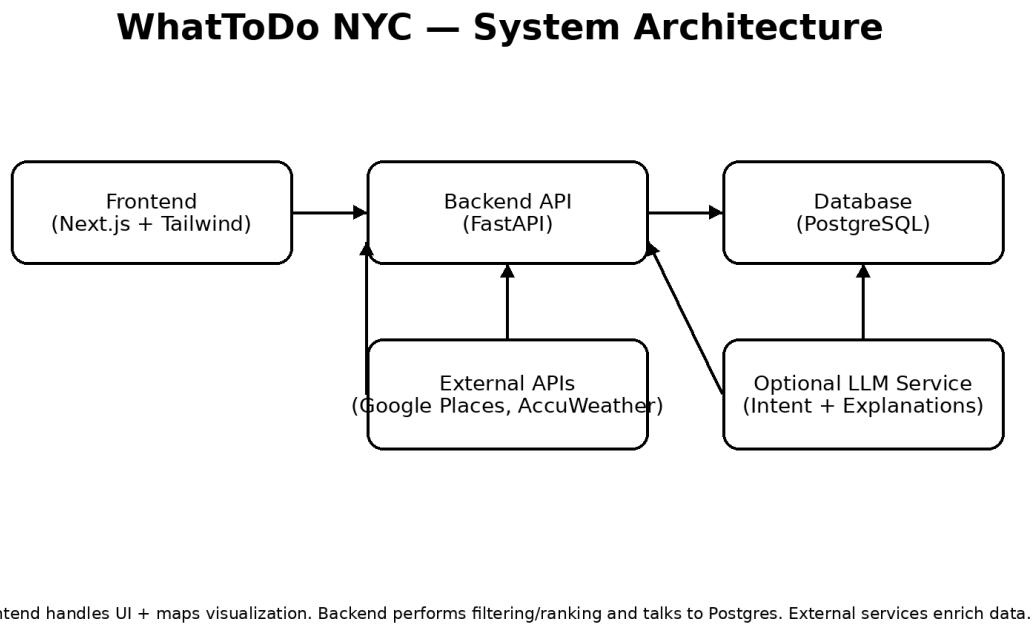

Figure 1: High-level architecture: Next.js UI calls FastAPI; FastAPI queries Postgres and enriches results via external APIs.

Seller/Promoter

A seller can claim or add a place and create time-bounded promotions that increase visibility without overriding relevance constraints.

Reviewer / Community

Users write reviews and cast authenticity votes (authentic vs touristy). These signals influence recommendation quality and transparency.

5 Recommendation Pipeline

1. **Candidate Retrieval**: fetch from Postgres cache; if insufficient, fetch via Google Places and cache
2. **Filtering**: open/closed hours, distance radius, budget range, category/tags
3. **Scoring**: weighted combination of relevance + distance + price fit + group fit + weather fit + review strength + authenticity + promotion boost
4. **Output**: return top-N results with short "why this" explanations

6 ER Diagram

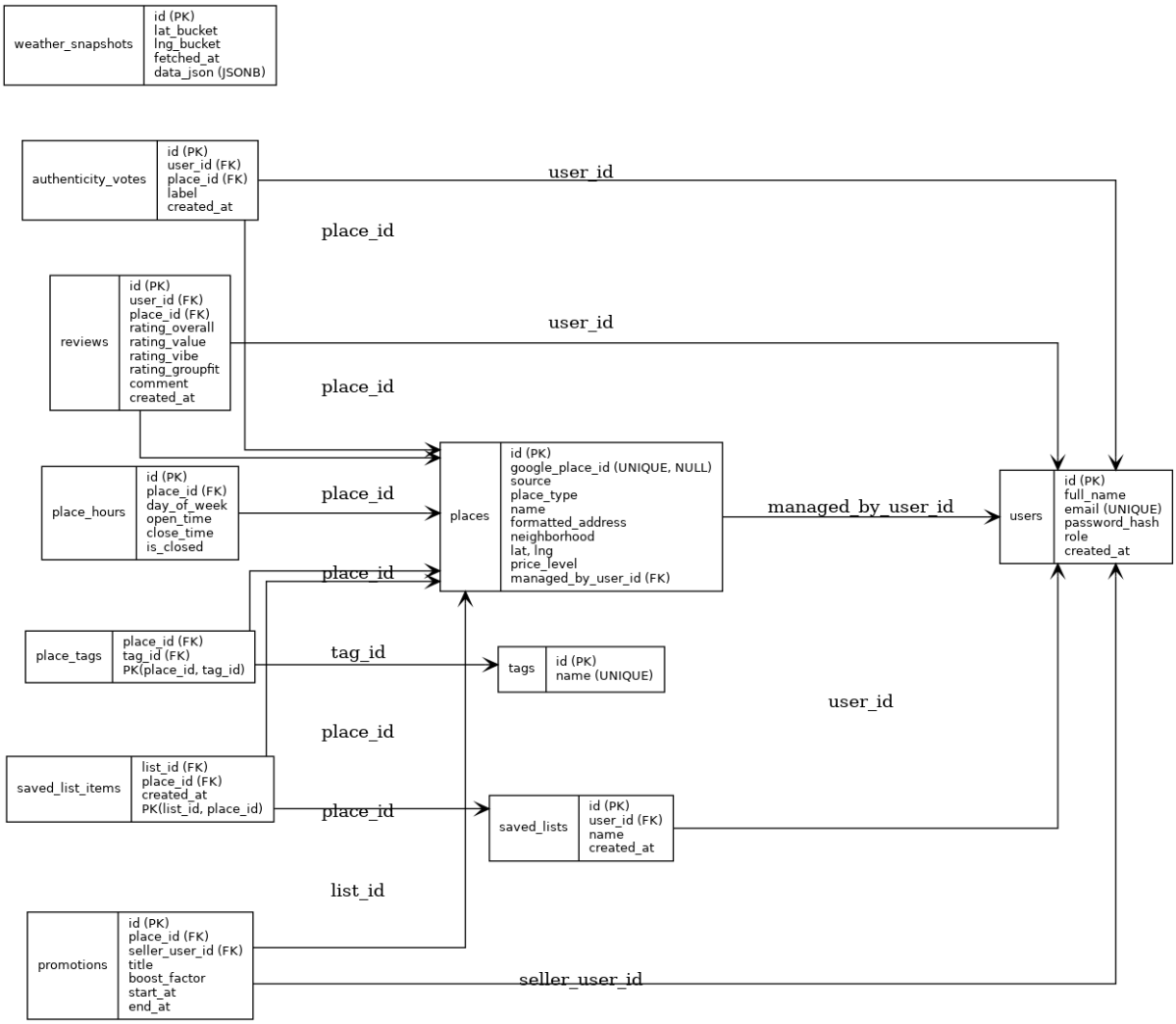

Figure 2: Entity Relationship Diagram (ERD) for the core schema. Boxes are tables; arrows indicate foreign keys.

7 Tables at a Glance

Table	Purpose
users	Registered accounts and roles (customer/reviewer/seller/admin).
places	Canonical listings (restaurants/events/activities) with location and seller management.
place_hours	Normalized weekly hours (one row per place per day).
reviews	User reviews with multi-dimensional ratings and optional text.
authenticity_votes	Per-user vote indicating authentic vs tourist-oriented for a place.
tags	Unique descriptive tags (e.g., cozy, good-for-groups).
place_tags	Join table mapping tags to places (many-to-many).
promotions	Seller promotions with time window and bounded boost.
weather_snapshots	Cached weather responses by geographic bucket + time.
saved_lists	Named lists owned by users (favorites/plans).
saved_list_items	Join table mapping saved lists to places.

8 Database Table Structures

The following subsections list each table's key columns, types, and important constraints (PK, FK, UNIQUE, CHECK). This matches the structure created in Supabase.

users

Column	Type	Notes / Constraints
id	UUID	PK
full_name	TEXT	
email	TEXT	UNIQUE
password_hash	TEXT	
role	user_role	ENUM
created_at	TIMESTAMPTZ	DEFAULT now()

places

Column	Type	Notes / Constraints
id	UUID	PK
google_place_id	TEXT	UNIQUE, nullable
source	place_source	ENUM
place_type	place_type	ENUM
name	TEXT	
formatted_address	TEXT	nullable
neighborhood	TEXT	nullable
lat	DOUBLE PRECISION	
lng	DOUBLE PRECISION	
price_level	INTEGER	1–4, nullable
phone	TEXT	nullable
website	TEXT	nullable
managed_by_user_id	UUID	FK→users.id, nullable
created_at	TIMESTAMPTZ	DEFAULT now()
updated_at	TIMESTAMPTZ	DEFAULT now()

place_hours

Column	Type	Notes / Constraints
id	UUID	PK
place_id	UUID	FK→places.id
day_of_week	SMALLINT	0–6
open_time	TIME	nullable
close_time	TIME	nullable
is_closed	BOOLEAN	DEFAULT false
(place_id, day_of_week)	—	UNIQUE

reviews

Column	Type	Notes / Constraints
id	UUID	PK
user_id	UUID	FK→users.id
place_id	UUID	FK→places.id
rating_overall	SMALLINT	1–5
rating_value	SMALLINT	1–5, nullable
rating_vibe	SMALLINT	1–5, nullable
rating_groupfit	SMALLINT	1–5, nullable
comment	TEXT	nullable
created_at	TIMESTAMPTZ	DEFAULT now()
(user_id, place_id)	—	UNIQUE

authenticity_votes

Column	Type	Notes / Constraints
id	UUID	PK
user_id	UUID	FK→users.id
place_id	UUID	FK→places.id
label	authenticity_label	ENUM
created_at	TIMESTAMPTZ	DEFAULT now()
(user_id, place_id)	—	UNIQUE

tags

Column	Type	Notes / Constraints
id	UUID	PK
name	TEXT	UNIQUE

place_tags

Column	Type	Notes / Constraints
place_id	UUID	FK→places.id
tag_id	UUID	FK→tags.id
(place_id, tag_id)	—	PK (composite)

promotions

Column	Type	Notes / Constraints
id	UUID	PK
place_id	UUID	FK→places.id
seller_user_id	UUID	FK→users.id
title	TEXT	
description	TEXT	nullable
boost_factor	NUMERIC(3,2)	1.00–3.00
start_at	TIMESTAMPTZ	
end_at	TIMESTAMPTZ	
created_at	TIMESTAMPTZ	DEFAULT now()

weather_snapshots

Column	Type	Notes / Constraints
id	UUID	PK
lat_bucket	DOUBLE PRECISION	
lng_bucket	DOUBLE PRECISION	
fetched_at	TIMESTAMPTZ	DEFAULT now()
data_json	JSONB	cached payload

saved_lists

Column	Type	Notes / Constraints
id	UUID	PK
user_id	UUID	FK→users.id
name	TEXT	
created_at	TIMESTAMPTZ	DEFAULT now()
(user_id, name)	—	UNIQUE

saved_list_items

Column	Type	Notes / Constraints
list_id	UUID	FK→saved_lists.id
place_id	UUID	FK→places.id
created_at	TIMESTAMPTZ	DEFAULT now()
(list_id, place_id)	—	PK (composite)

9 Integrity and Design Notes

- **FK constraints** enforce valid relationships (e.g., reviews must reference existing users and places).
- **Uniqueness** rules prevent duplicates (one review per user/place; one authenticity vote per user/place).
- **Normalized hours** supports correct open/closed logic without repeating fields.
- **Join tables** (`place_tags`, `saved_list_items`) correctly model many-to-many relationships.

10 Deployment

- **Database**: Supabase Postgres
- **Backend**: Render / Fly.io (FastAPI)
- **Frontend**: Vercel (Next.js)